



Image Restoration

Restorasi citra

Bertujuan untuk merekonstruksi kembali citra yang mengalami distorsi (*distorted image*) atau degradasi (*degraded image*) menjadi bentuk citra semula (*original image*) berdasarkan model yang ideal.

Modelkan degradasi (*original image* $\rightarrow$ *distorted image*), kemudian lakukan proses berkebalikan (*inverse*) untuk merekonstruksi citra semula (*distorted image* $\rightarrow$ *original image*)

Citra hasil rekonstruksi sedapat mungkin menghasilkan citra yang mendekati citra semula. Jadi, citra hasil rekonstruksi merupakan estimasi citra semula.

Image Restoration vs Image Enhancement

Enhancement

- Berkaitan dengan ekstraksi fitur citra (kontras, kecerahan, histogram, dll)
- Sulit mengkuantisasi kinerja metodenya
- Lebih subyektif; membuat citra “tampil lebih baik (*look better*)”

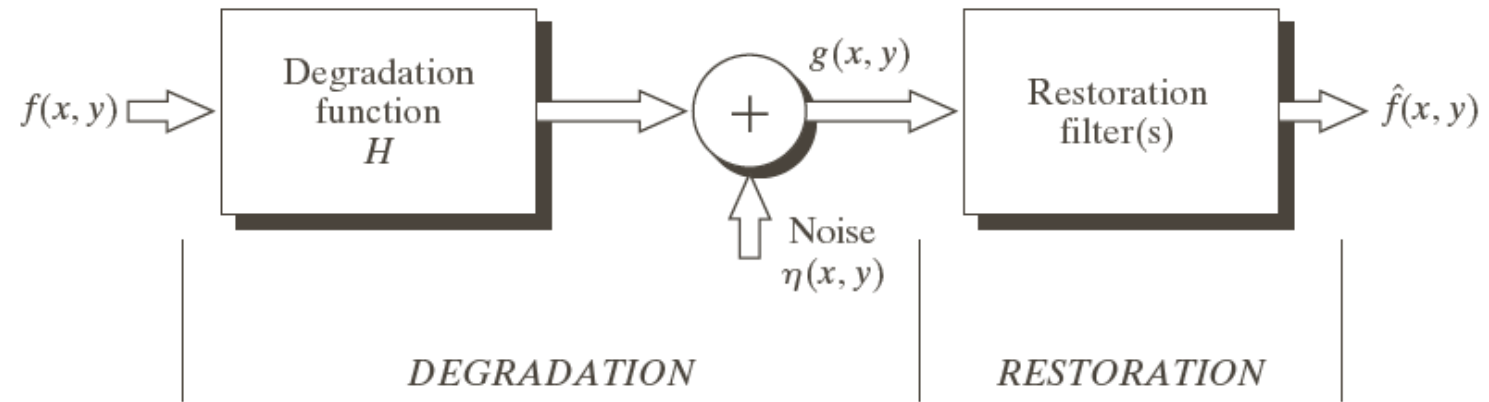
Restoration

- Berkaitan dengan pemulihan citra yang mengalami degradasi
- Kinerja metode dapat dikuantisasi
- Lebih obyektif; merekonstruksi menjadi citra semula

Model Proses Degradasi/Restorasi Gambar

FIGURE 5.1

A model of the image degradation/restoration process.



► Degradation

- Degradation function H
- Additive noise $\eta(x, y)$

Model Proses Degradasi/Restorasi Gambar

If H is a linear, position-invariant process, then the degraded image is given in the spatial domain by

$$g(x, y) = h(x, y) \star f(x, y) + \eta(x, y)$$

The model of the degraded image is given in the frequency domain by

$$G(u, v) = H(u, v)F(u, v) + N(u, v)$$

Faktor-faktor penyebab distorsi pada citra

Sumber utama degradasi Adalah selama akuisisi dan/atau transmisi gambar

- Degradasi citra akibat kondisi lingkungan selama proses transmisi citra

Contoh: petir, turbulensi atmosfer

- Derau pada citra akibat kualitas penginderaan pada peralatan sensor (**sensor noise**)

Contoh: level cahaya dan temperatur sensor pada kamera CCD

- *Blurring* pada citra diakibatkan oleh sensor

Contoh: kamera bergerak atau out-of-focus

- Distorsi geometrik

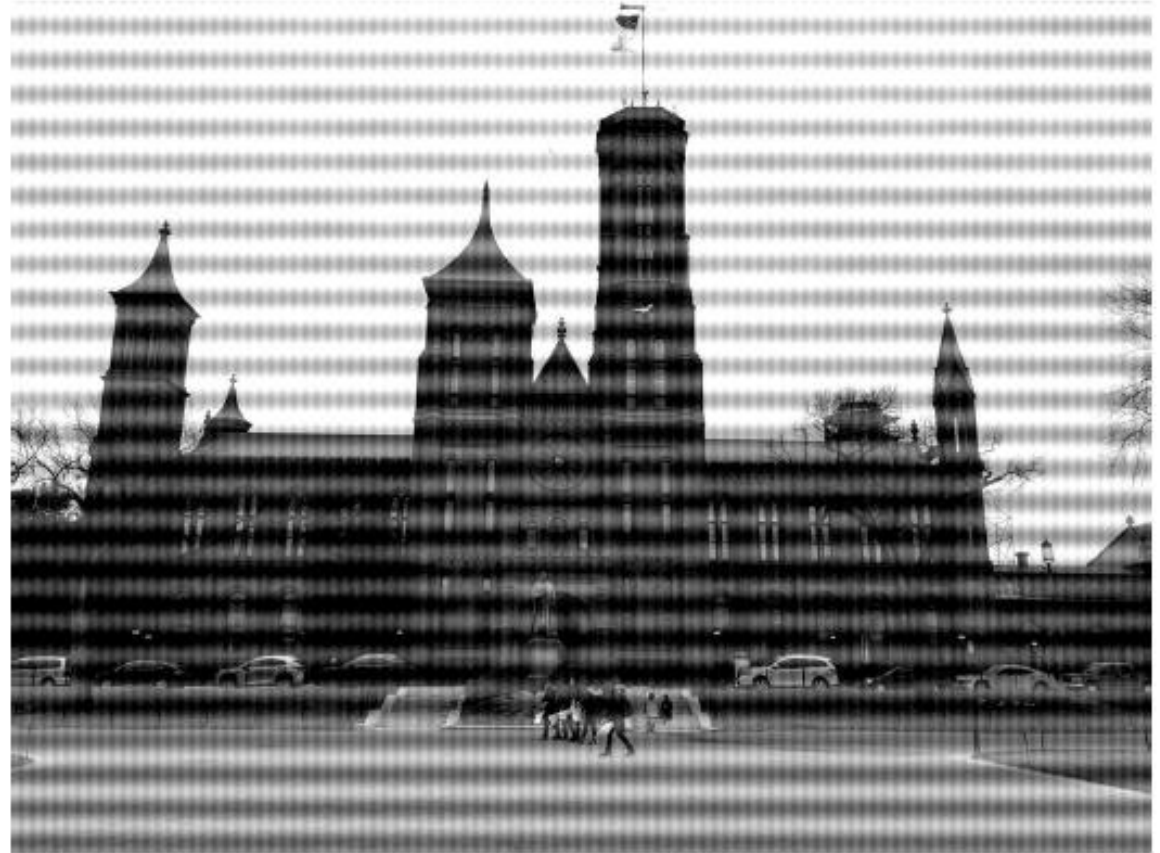
Contoh: foto bumi yang diambil oleh kamera dari satelit

Image Denoising

- *Image denoising* adalah proses pengurangan bintik-bintik (noise) visual yang tidak diinginkan, seperti *noise* Gaussian, *salt-and-pepper*, atau *speckle* dari citra, untuk menghasilkan versi yang lebih bersih dan jelas.
- Mengapa kita ingin menghilangkan noise?
 - ❖ Secara visual tidak menyenangkan
 - ❖ Buruk untuk kompresi
 - ❖ Buruk untuk analisis

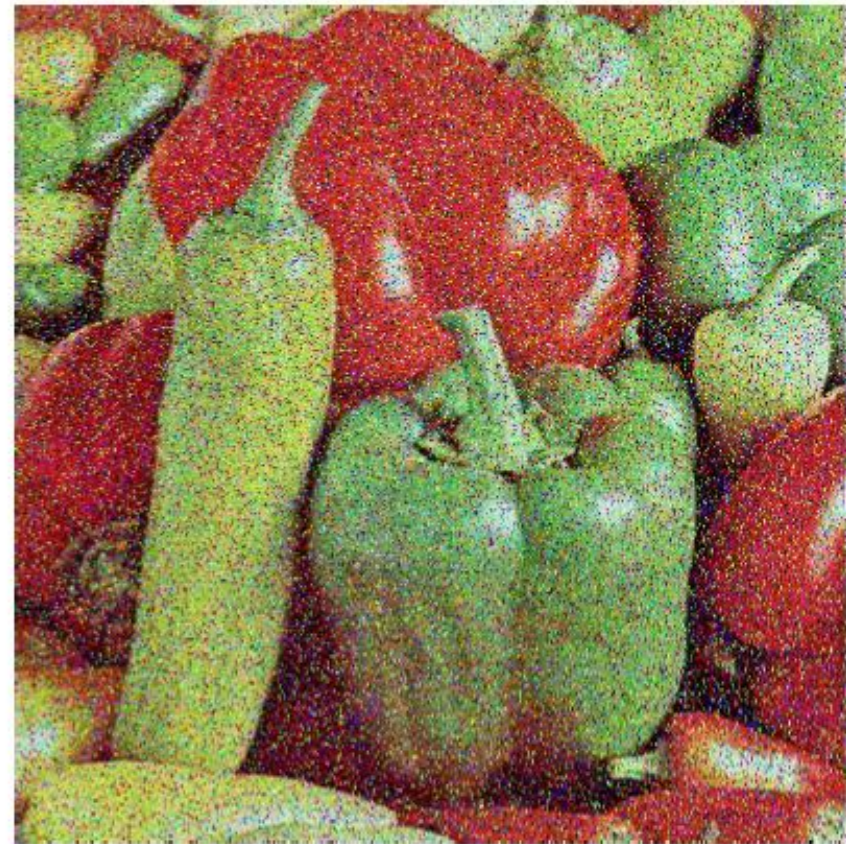


blurred image



Citra yang mengalami derau periodik

noisy image (salt and peppers)



Noisy image (salt and pepper noise)



Citra yang mengalami turbulensi atmosfer

Contoh Citra Ber-noise



thermal imaging



ultrasound imaging



electrical interference



physical interference

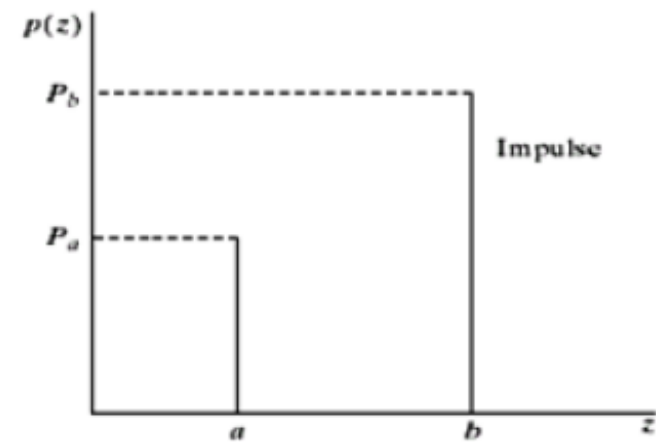
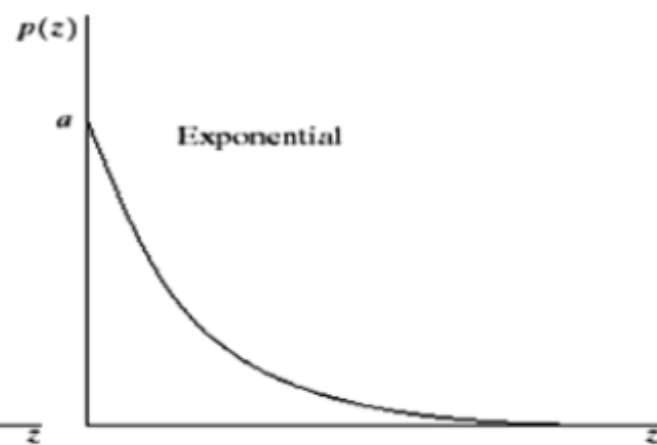
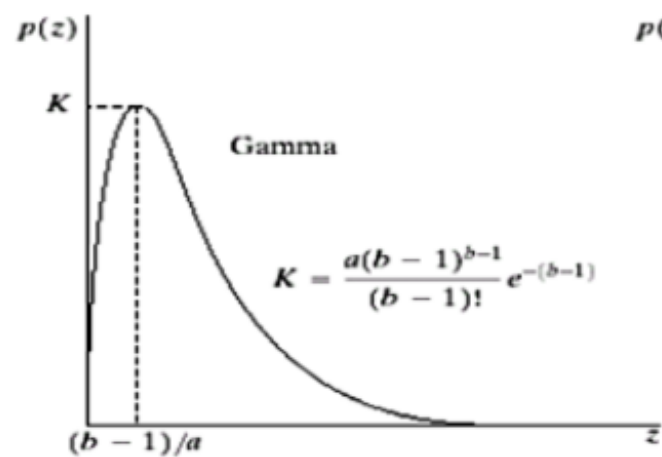
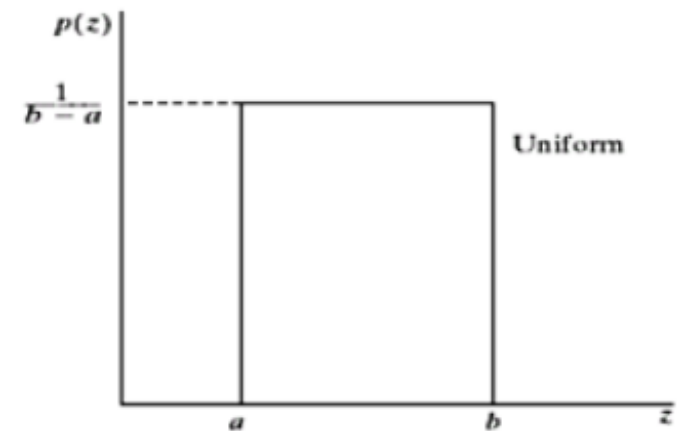
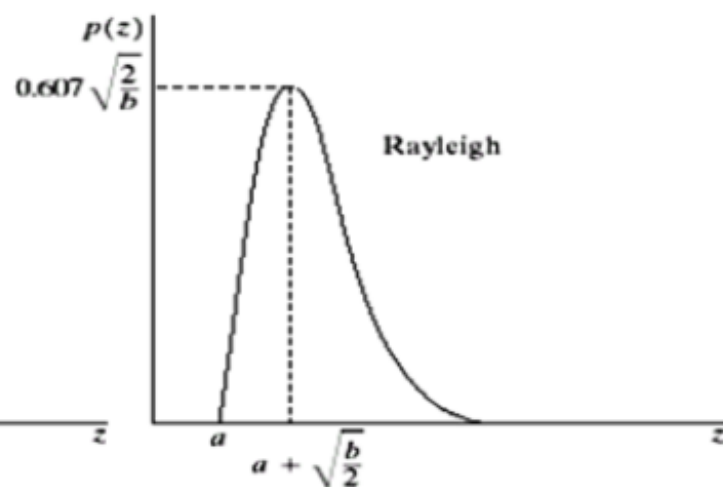
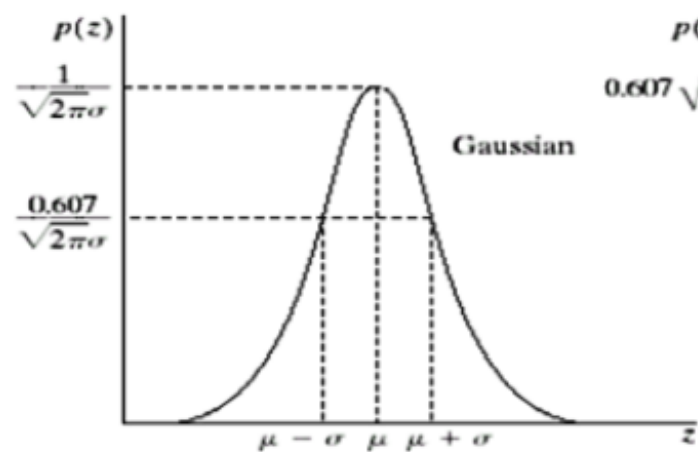
Pemodelan Derau (noise)

Asumsikan derau:

- independen dari koordinat spasial
- tidak berkorelasi dengan konten citra

Derau dianggap sebagai variabel acak. Variabel acak tersebut dicirikan oleh fungsi kepadatan peluang (*probability densityfunction*) atau PDF.

Beberapa PDF yang umum ditemukan di dalam aplikasi pengolahan citra adalah: *Gaussian noise, Rayleigh noise, Erlang (gamma) noise, exponential noise, uniform noise, dan impulse (salt & pepper) noise.*



- **Gaussian noise**

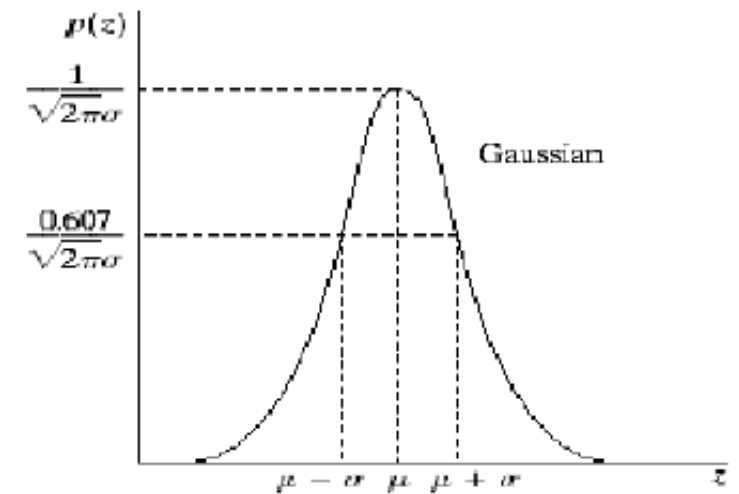
- Probability density function (PDF)

$$p(z) = \frac{1}{\sqrt{2\pi}\sigma} e^{-(z-\mu)^2 / 2\sigma^2}$$

- z : gray level (Gaussian random variable)
- μ : mean of average value of z
- σ : standard deviation of z
- σ^2 : variance of z

- **PDF of Gaussian noise**

- 70% of z in $[\mu - \sigma, \mu + \sigma]$
- 90% of z in $[\mu - 2\sigma, \mu + 2\sigma]$



Impulse Noise (salt-pepper Noise)

Definition

Setiap piksel dalam gambar memiliki probabilitas $p/2$ ($0 < p < 1$) menjadi terkontaminasi oleh titik putih (garam) atau titik hitam (merica)

$$Y(i, j) = \begin{cases} 255 & \text{dengan probabilitas } p/2 \\ 0 & \text{dengan probabilitas } p/2 \\ X(i, j) & \text{with probability of } 1-p \end{cases} \begin{matrix} \diagup \\ \diagdown \\ \text{---} \end{matrix} \begin{matrix} \text{noisy pixels} \\ \text{noisy pixels} \\ \text{clean pixels} \end{matrix}$$

$1 \leq i \leq H, 1 \leq j \leq W$ X: citra bebas noise, Y: citra ber-noise

Catatan: dalam beberapa aplikasi, piksel noise tidak hanya hitam atau putih, yang membuat masalah penghapusan noise impuls lebih sulit

Contoh Numerik

P=0.1

128	128	128	128	128	128	128	128	128	128
128	128	128	128	128	128	128	128	128	128
128	128	128	128	128	128	128	128	128	128
128	128	128	128	128	128	128	128	128	128
128	128	128	128	128	128	128	128	128	128
128	128	128	128	128	128	128	128	128	128
128	128	128	128	128	128	128	128	128	128
128	128	128	128	128	128	128	128	128	128
128	128	128	128	128	128	128	128	128	128
128	128	128	128	128	128	128	128	128	128

X

128	128	255	0	128	128	128	128	128	128
128	128	128	128	0	128	128	128	128	0
128	128	128	128	128	128	128	128	128	128
128	128	0	128	128	128	128	128	128	128
128	128	128	128	128	128	128	128	128	128
128	128	128	128	128	128	128	128	128	128
128	128	128	128	128	128	128	128	128	128
128	128	128	128	128	128	128	128	128	128
0	128	128	128	128	255	128	128	128	128
128	128	128	128	128	128	128	128	128	255
128	128	128	128	128	128	128	255	128	128

Y

Tingkat derau $p = 0,1$ berarti bahwa sekitar 10% piksel terkontaminasi oleh kebisingan salt/papper (disorot dengan warna hijau)

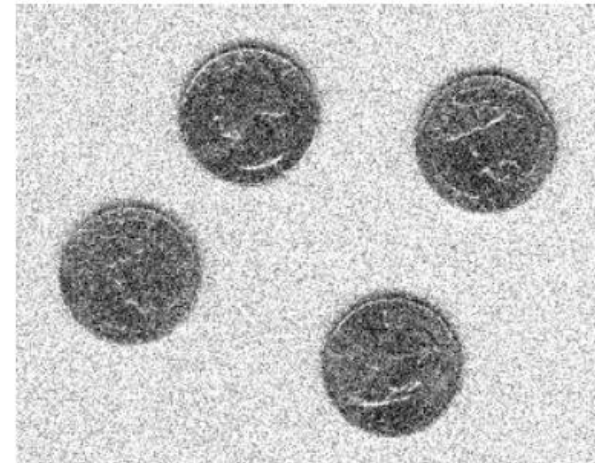
Membuat citra yang mengandung derau *gaussian* dengan Matlab

```
A = imread('eight.png');  
B = imnoise(A,'gaussian',0, 0.02); % mean = 0.0, variansi = 0.02  
imshow(A), title('original image');  
figure, imshow(B), title('noisy image (gaussian)');
```

original image



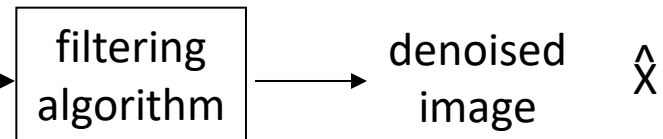
noisy image (gaussian)



Masalah Penghapusan Derau Impuls



Noisy image Y



Bisakah kita membuat citra $\hat{X}$ yang didebisingkan sedekat mungkin dengan citra X bebas noise?

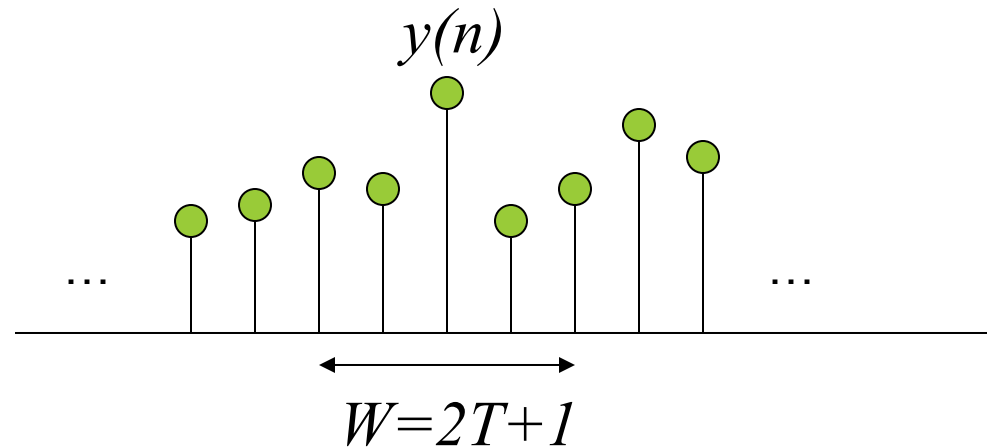
Median Operator

Given a sequence of numbers $\{y_1, \dots, y_N\}$

- Mean: average of N numbers
- Min: minimum of N numbers
- Max: maximum of N numbers
- Median: half-way of N numbers

Example $\vec{y} = [50, 0, 52, 255, 54, 55, 56]$
 $\downarrow$
 sorted $\vec{y} = [0, 50, 52, 54, 55, 56, 255]$
 $\downarrow$
 $median(\vec{y}) = 54$

1D Median Filtering



$$\hat{x}(n) = \text{median}[y(n-T), \dots, y(n), \dots, y(n+T)]$$

MATLAB command: `x=median(y(n-T:n+T));`

Note: median operator is **nonlinear**

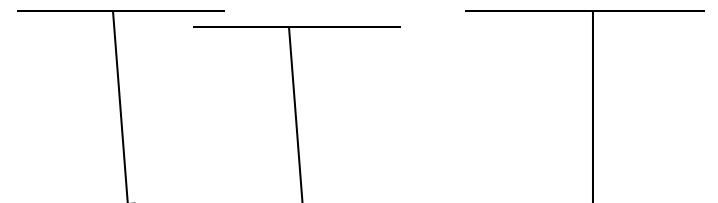
Numerical Example

T=1:

$$\vec{y} = [50, 0, 52, 255, 54, 55, 56]$$

Boundary Padding

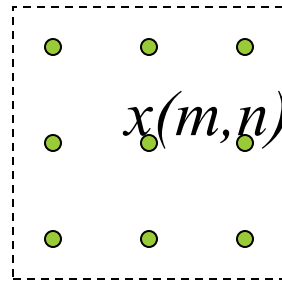
$$\vec{y} = [\mathbf{50}, 50, 0, 52, 255, 54, 55, 56, \mathbf{56}]$$



The diagram shows three horizontal arrows pointing downwards from the padded vector $\vec{y}$ to the vector $\hat{x}$. The first arrow points from the first element (50) of $\vec{y}$ to the first element (50) of $\hat{x}$. The second arrow points from the third element (0) of $\vec{y}$ to the third element (52) of $\hat{x}$. The third arrow points from the sixth element (54) of $\vec{y}$ to the sixth element (55) of $\hat{x}$.

$$\hat{x} = [50, 50, 52, 54, 54, 55, 55]$$

2D Median Filtering



W : $(2T+1)$ -by- $(2T+1)$ window

$$\hat{x}(m, n) = \text{median}[y(m-T, n-T), \dots, y(m-T, n+T), \dots, \\ y(m, n), \dots, y(m+T, n-T), \dots, y(m+T, n+T)]$$

MATLAB command: `x=medfilt2(y,[2*T+1,2*T+1]);`

Numerical Example

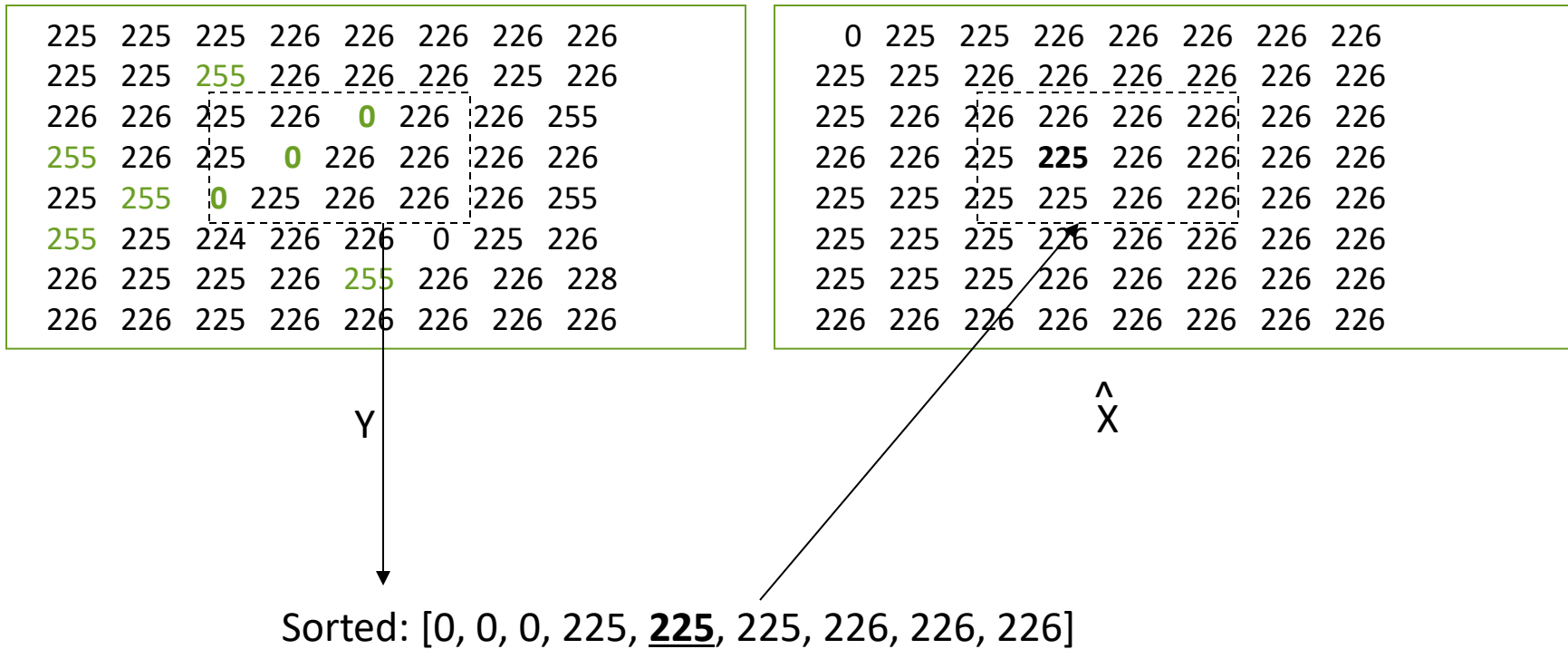
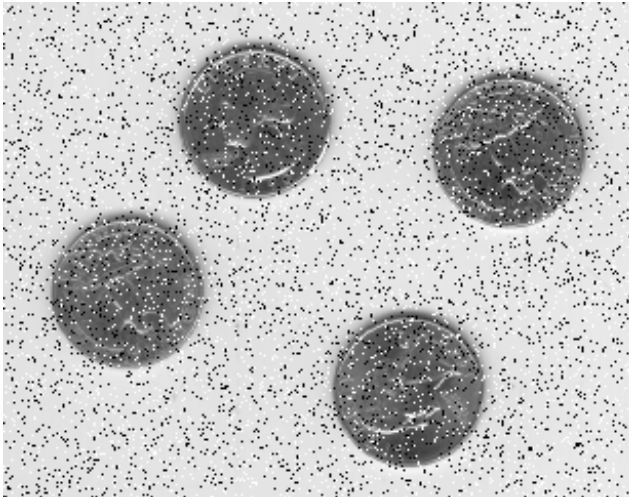


Image Example

$P=0.1$

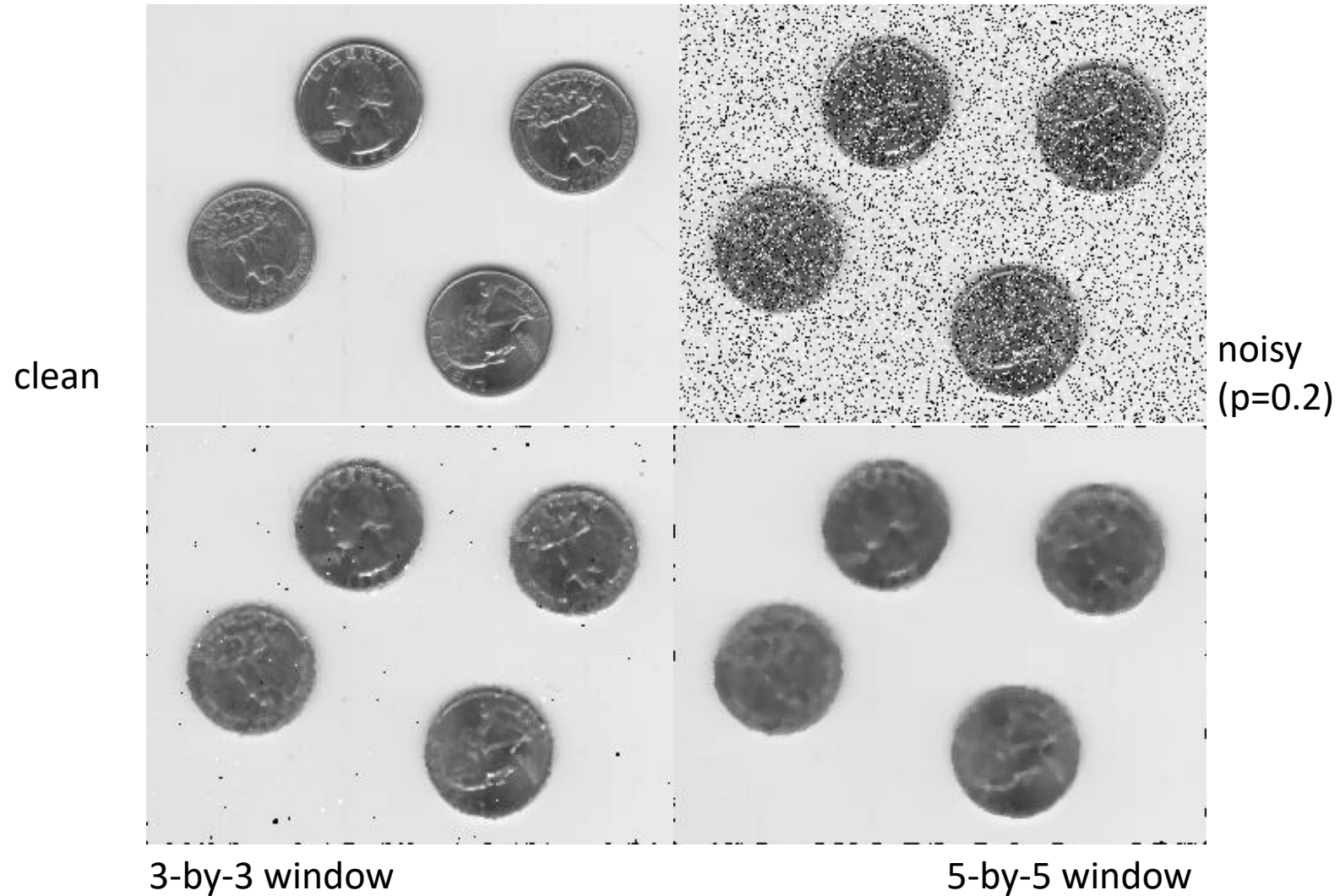


Noisy image Y



Denoised image $\hat{X}$
3-by-3 window

Image Example (Con't)



Refleksi

Apa yang baik tentang operasi median?

Karena kita tahu derau impuls muncul sebagai titik hitam (minimum) atau putih (maksimum), filtermedian secara efektif menekan derau

Apa yang buruk tentang operasi median?

- ✓ Ini juga memengaruhi piksel bersih
- ✓ Keburaman tepi yang terlihat setelah pemfilteran median

Ide Meningkatkan Filter Median

Can we get rid of impulse noise without affecting clean pixels?

- Yes, if we know **where** the clean pixels are or equivalently where the noisy pixels are

How to detect noisy pixels?

- They are black or white dots

Median Filtering with Noise Detection

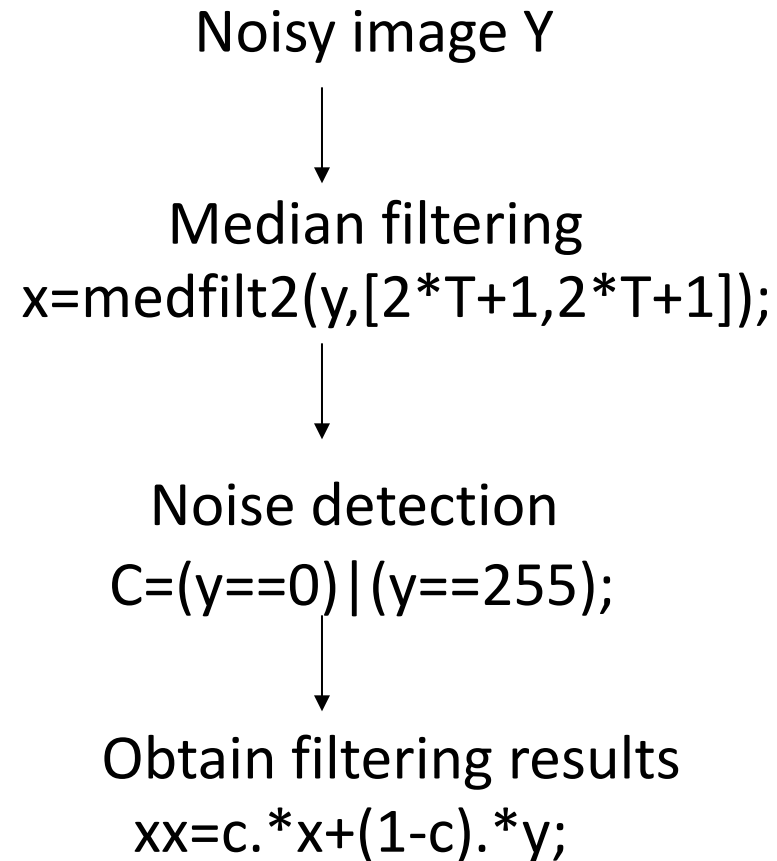


Image Example

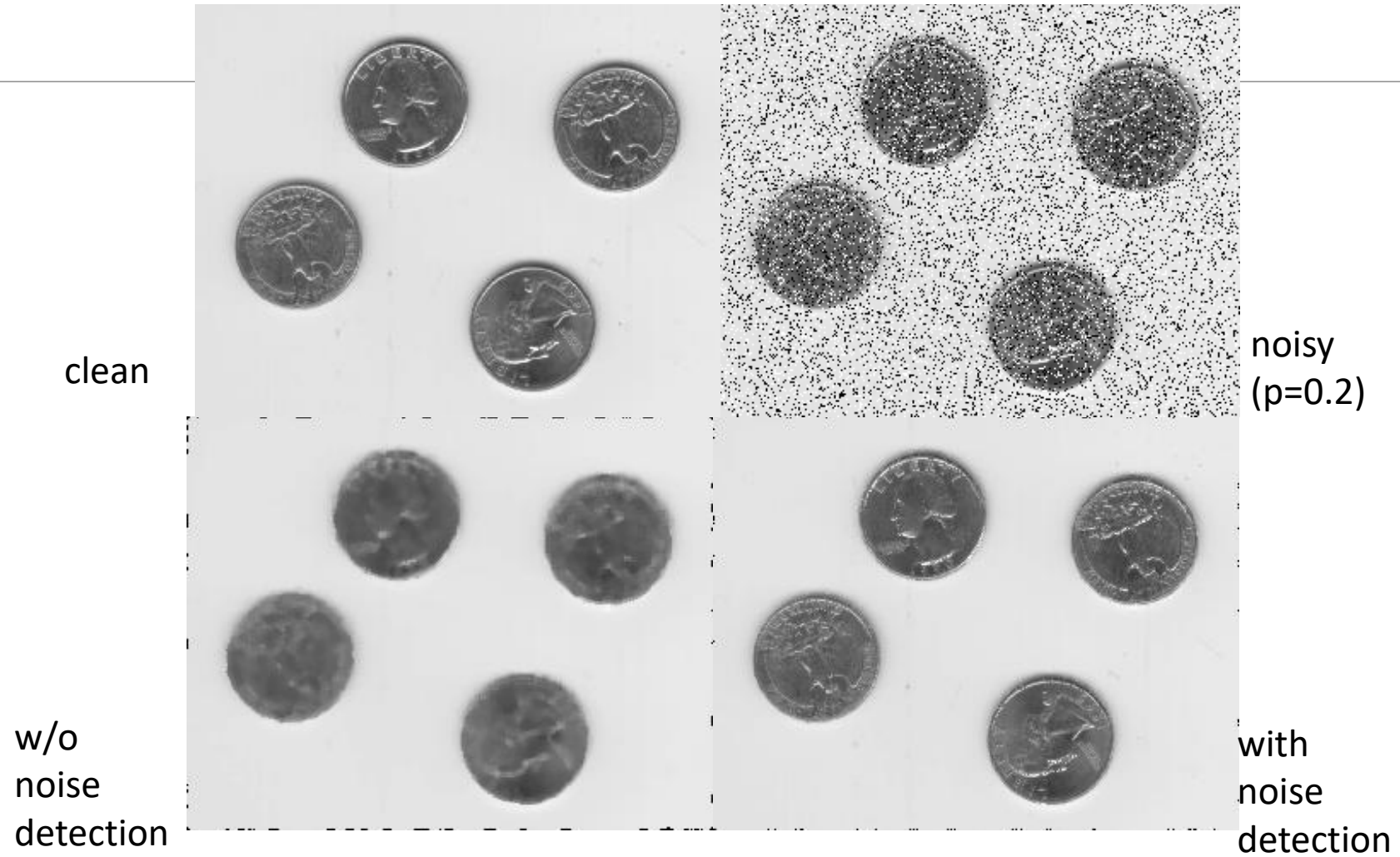


Image Denoising

- Introduction
- Impulse noise removal
 - Median filtering
- Additive white Gaussian noise removal
 - 2D convolution and DFT
- Periodic noise removal
 - Band-rejection and Notch filter

Additive White Gaussian Noise

Definition

Each pixel in an image is disturbed by a Gaussian random variable
With zero mean and variance σ^2

$$Y(i, j) = X(i, j) + N(i, j),$$

$$N(i, j) \sim N(0, \sigma^2), 1 \leq i \leq H, 1 \leq j \leq W$$

X: noise-free image, Y: noisy image

Note: unlike impulse noise situation, every pixel in the image
Contaminated by AWGN is noisy

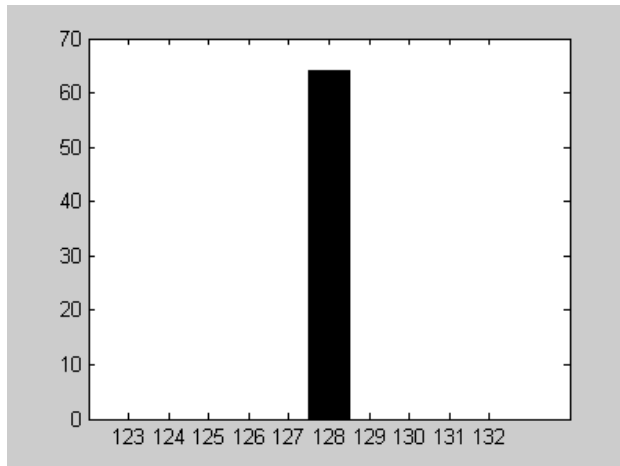
Numerical Example

128	128	128	128	128	128	128	128
128	128	128	128	128	128	128	128
128	128	128	128	128	128	128	128
128	128	128	128	128	128	128	128
128	128	128	128	128	128	128	128
128	128	128	128	128	128	128	128
128	128	128	128	128	128	128	128
128	128	128	128	128	128	128	128

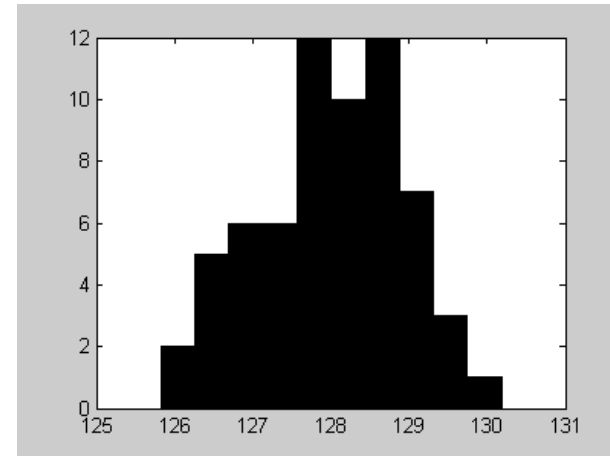
$$\sigma^2 = 1$$

128	128	129	127	129	126	126	128
126	128	128	129	129	128	128	127
128	128	128	129	129	127	127	128
128	129	127	126	129	129	129	128
127	127	128	127	129	127	129	128
129	130	127	129	127	129	130	128
129	128	129	128	128	128	129	129
128	128	130	129	128	127	127	126

X



Y



MATLAB Command

`>Y = IMNOISE(X,'gaussian',m,v)`

or

`>Y = X+m+randn(size(X))*v;`

Note: `rand()` generates random numbers uniformly distributed over [0,1]

`randn()` generates random numbers observing Gaussian distribution
 $N(0,1)$



Restorasi citra dalam ranah frekuensi

Fourier Series

forward $F(w) = \sum_{-\infty}^{\infty} f(n)e^{-jwn}$

inverse $f(n) = \frac{1}{2\pi} \int_{-\pi}^{\pi} F(w)e^{jwn} dw$

time-domain convolution

$$f(n) \otimes h(n)$$

frequency-domain multiplication

$$F(w)H(w)$$

Note that the input signal is a **discrete** sequence
while its FT is a **continuous** function

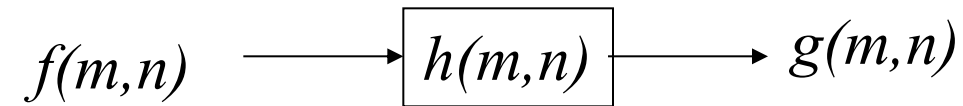
Filtering in the Frequency Domain

$$f(n) \longrightarrow \boxed{h(n)} \longrightarrow g(n) \qquad F(k) \longrightarrow \boxed{H(k)} \longrightarrow G(k)$$

$$g(n) = f(n) \otimes h(n) \xrightarrow{\text{DFT}} G(k) = F(k)H(k)$$

convolution in the **time** domain is equivalent to multiplication in the **frequency** domain

2D Linear Filtering



$$g(m,n) = \sum_{k,l=-\infty}^{\infty} h(k,l) f(m-k, n-l) = h(m,n) \otimes f(m,n)$$

↓

2D convolution

MATLAB function: $C = \text{CONV2}(A, B)$

Fourier Series (2D case)

$$F(w_1, w_2) = \sum_{m=-\infty}^{\infty} \sum_{n=-\infty}^{\infty} f(m, n) e^{-j(w_1 m + w_2 n)}$$

spatial-domain convolution

$$f(m, n) \otimes h(m, n)$$

frequency-domain multiplication

$$F(w_1, w_2) H(w_1, w_2)$$

Note that the input signal is **discrete**
while its FT is a **continuous** function

Filter Examples

Low-pass (LP)

1D

$$h^1(n)=[1,1]$$



$$|h^1(w)|=2\cos(w/2)$$



2D

$$h(n)=[1,1;1,1]$$



$$|h(w_1, w_2)|=4\cos(w_1/2)\cos(w_2/2)$$

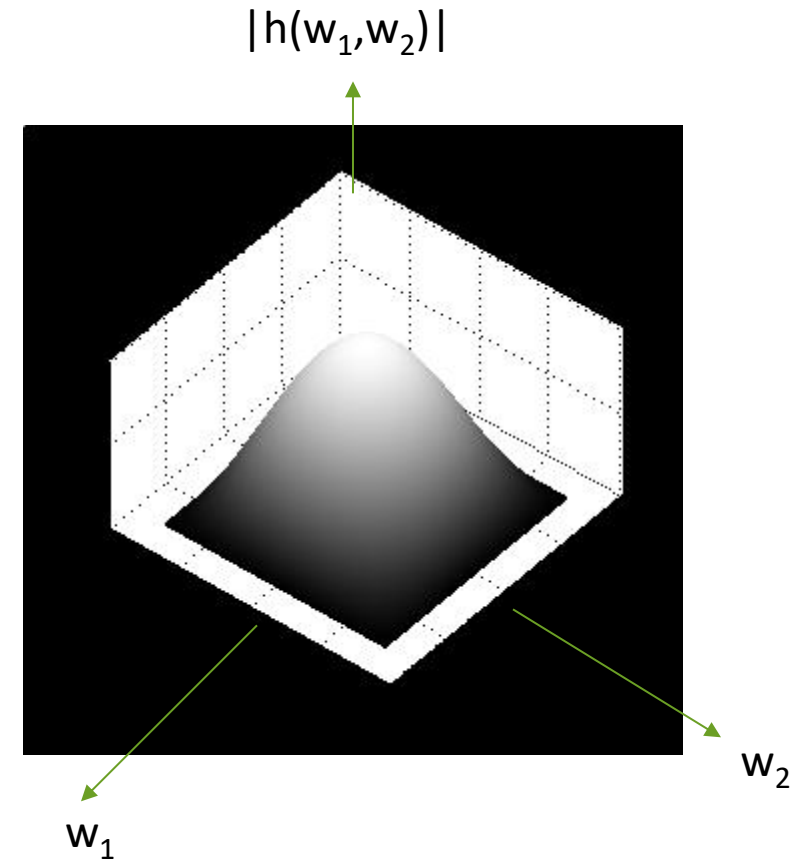
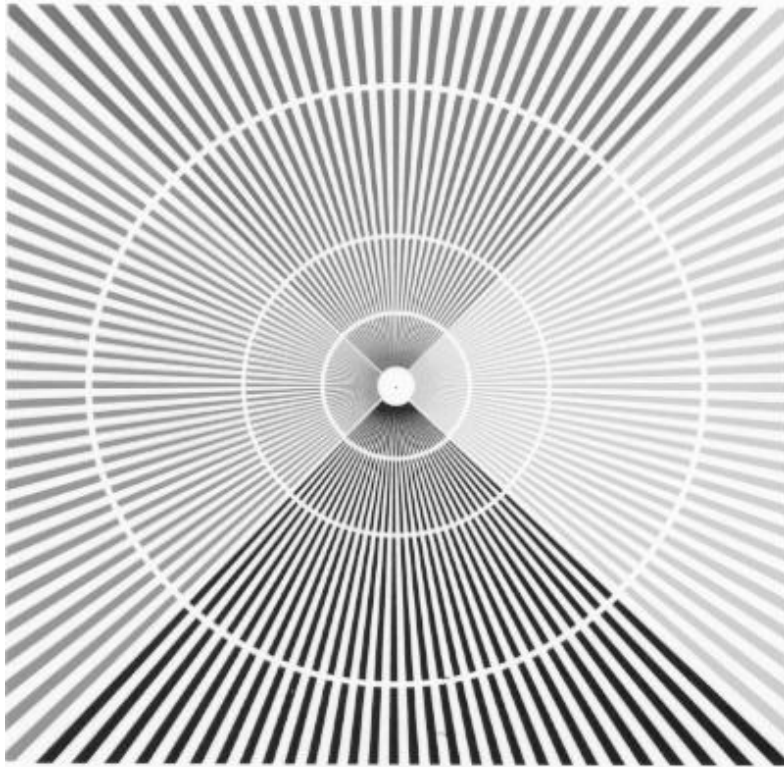
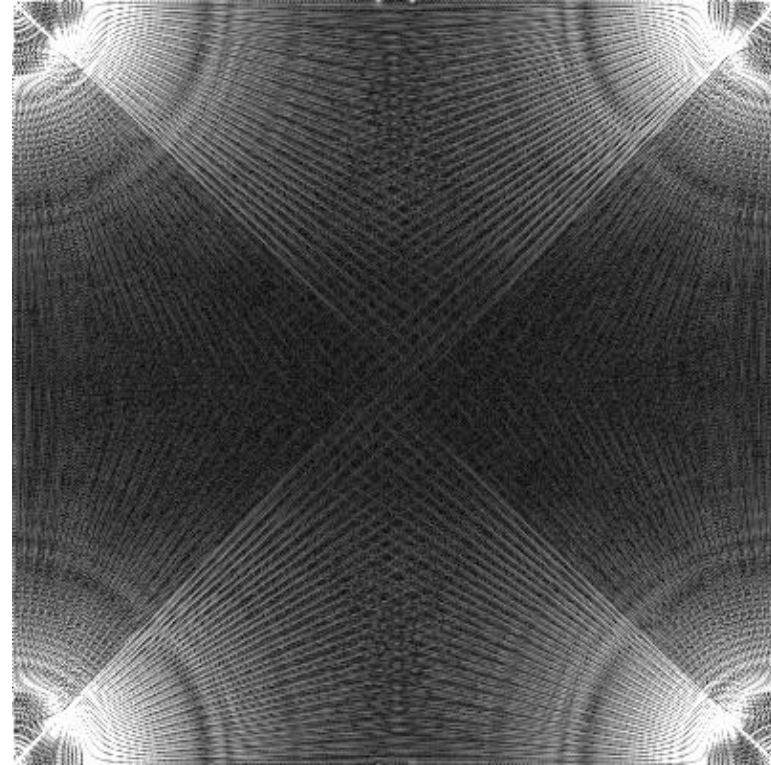


Image DFT Example

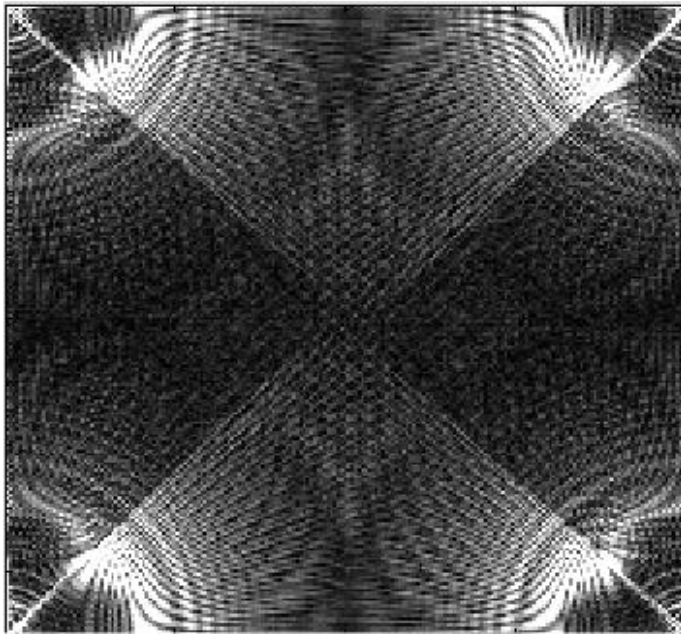


Original ray image $\mathbf{X}$

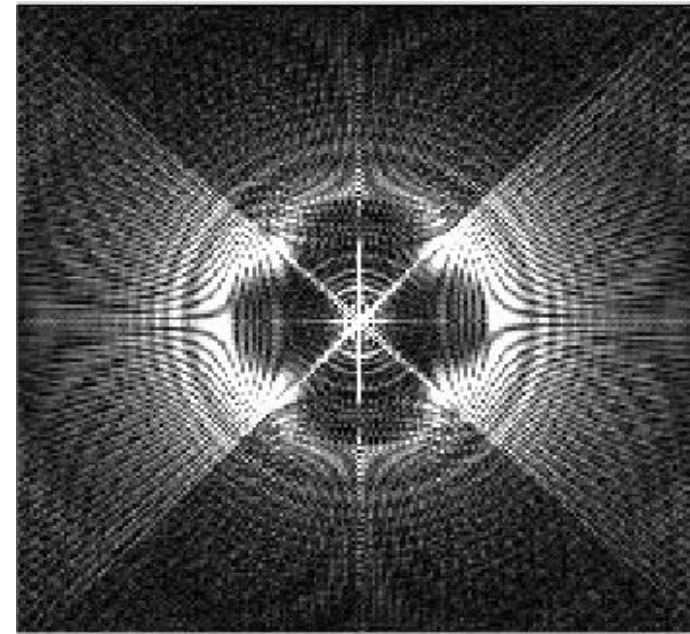


choice 1: $\mathbf{Y}=\text{fft2}(\mathbf{X})$

Image DFT Example (Con't)



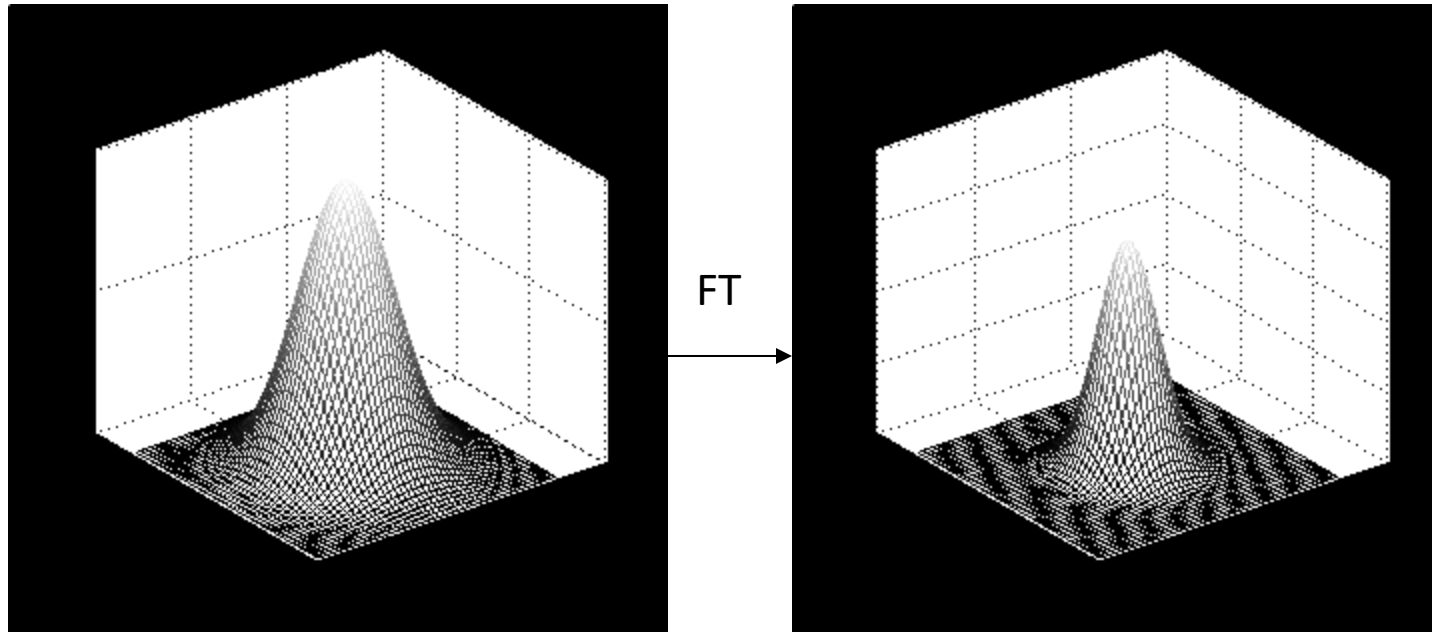
choice 1: $Y = \text{fft2}(X)$
Low-frequency at four corners



choice 2: $Y = \text{fftshift}(\text{fft2}(X))$
Low-frequency at the center

FFTSHIFT Shift zero-frequency component to center of spectrum.

Gaussian Filter



$$h(m, n) = \exp\left(-\frac{m^2 + n^2}{2\sigma^2}\right)$$

$$H(w_1, w_2) = \exp\left(-\frac{w_1^2 + w_2^2}{2\sigma^2}\right)$$

MATLAB code: `>h=fspecial('gaussian', HSIZE,SIGMA);`

Image Example

noisy



$PSNR=20.2dB$
($\sigma=25$)

denoised



$PSNR=24.4dB$
($\sigma=1$)

denoised



$PSNR=22.8dB$
($\sigma=1.5$)

Matlab functions: imfilter, filter2

Experimental Results

noisy



$PSNR=20.2dB$
($\sigma=25$)

linear diffusion



$PSNR=24.4dB$
(Gaussian filtering)

nonlinear diffusion



$PSNR=27.5dB$
(TV filtering)

Hammer-Nail Analogy

Gaussian filter



median filter



???



salt-pepper/
impulse noise



Gaussian noise



periodic noise

Image Denoising

- Introduction
- Impulse noise removal
 - Median filtering
- Additive white Gaussian noise removal
 - 2D convolution and DFT
- Periodic noise removal
 - Band-rejection and Notch filter

Restorasi citra dalam ranah frekuensi digunakan untuk mengestimasi citra yang terdistorsi akibat:

- derau periodik
- motion blur

Periodic Noise

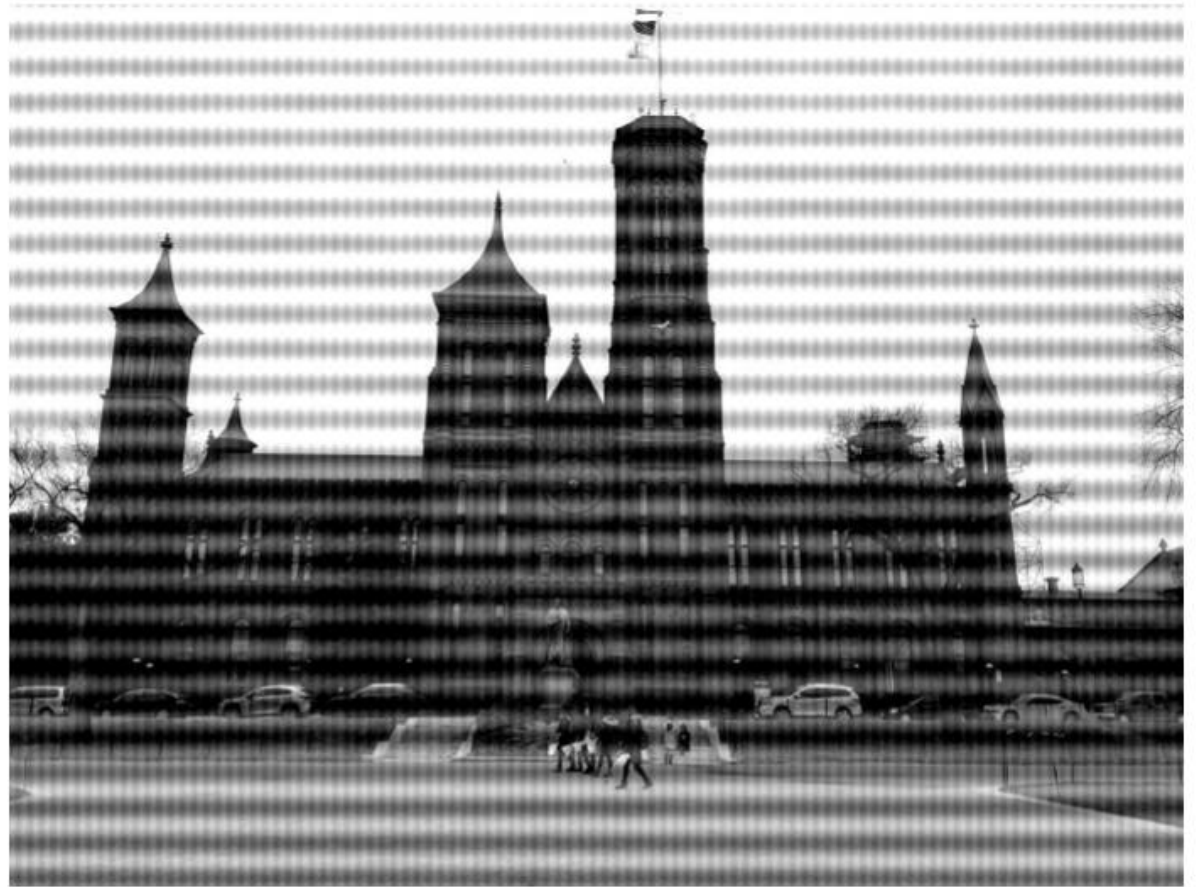
Source: electrical or electromechanical interference during image acquisition

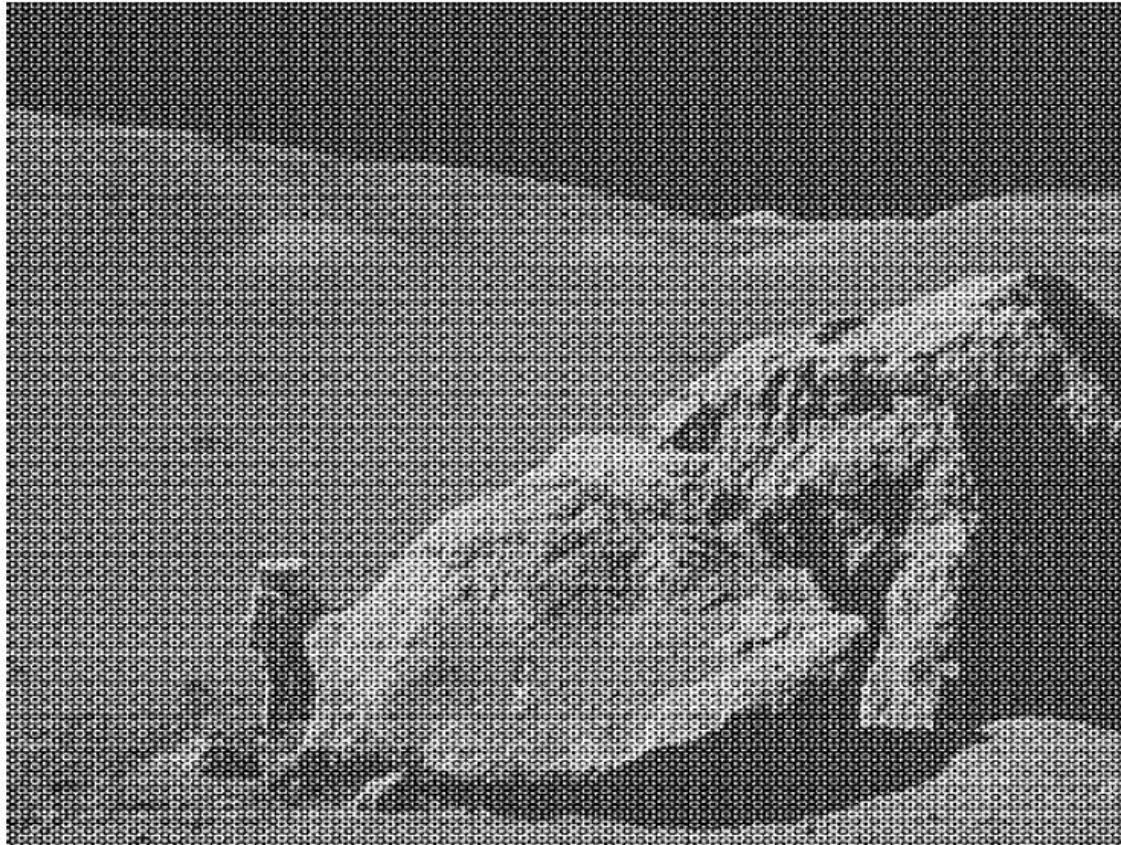
Characteristics

- Spatially dependent
- Periodic – easy to observe in frequency domain

Processing method

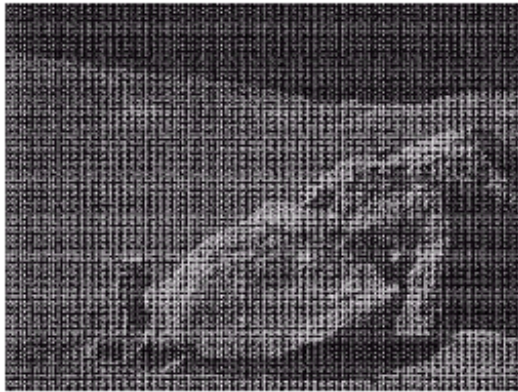
- Suppressing noise component in frequency domain





Citra dengan derau periodik lainnya

Image Example



spatial



Frequency (note the four pairs of bright dots)

Program Matlab untuk menambahkan derau periodic ke dalam citra

```
I = imread('lena.bmp');  
J = rgb2gray(I);  
s = size(J);  
[x,y] = meshgrid(1:s(1), 1:s(2));  
p = sin(x/3+y/5)+1;  
J_noise= (im2double(J)+ p'/2)/2;  
imshow(J);  
figure, imshow(J_noise);
```



Reduksi Derau Periodik

Menggunakan beberapa macam penapis:

- Band reject filters
- Bandpass filters
- Notch filters

❖ *Bandreject, bandpass dan notch filters can be used for periodic noise removal.*

❖ *Bandreject filters remove/attenuate a band of frequencies about the origin of the Fourier transform.*

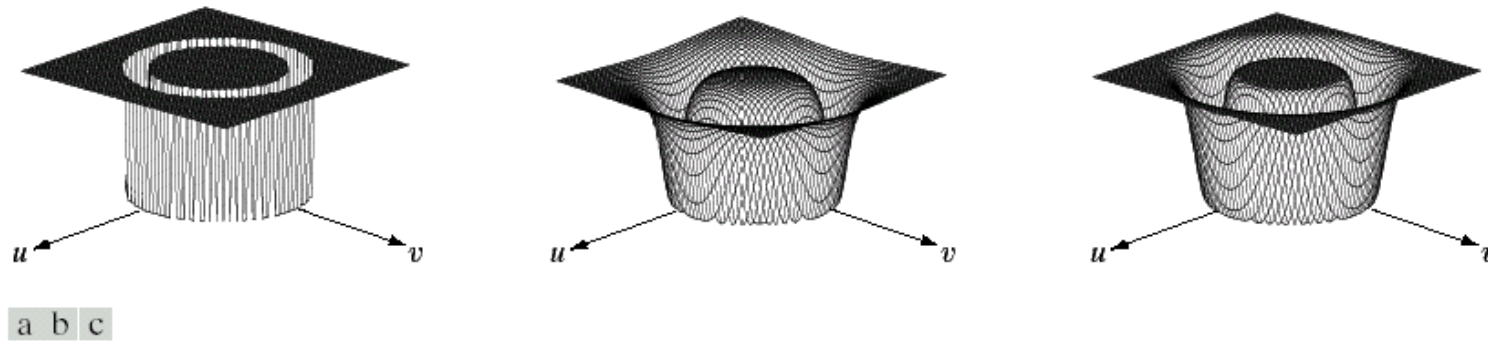


FIGURE 5.15 From left to right, perspective plots of ideal, Butterworth (of order 1), and Gaussian bandreject filters.

Bandreject filter

• An Ideal Bandreject filter is given by:

$$H(u, v) = \begin{cases} 1 & \text{if } D(u, v) < D_0 - \frac{W}{2} \\ 0 & \text{if } D_0 - \frac{W}{2} \leq D(u, v) \leq D_0 + \frac{W}{2} \\ 1 & \text{if } D(u, v) \geq D_0 + \frac{W}{2} \end{cases}$$

- W is the width of the band
- D_0 is the radial center
- $D(u, v)$ distance from the origin.

• Butterworth Bandreject filter is given by:

$$H(u, v) = \frac{1}{1 + \left[\frac{D(u, v)W}{D^2(u, v) - D_0^2} \right]^{2n}}$$

- W is the width of the band
- D_0 is the radial center
- $D(u, v)$ distance from the origin.
- n is the order of the filter

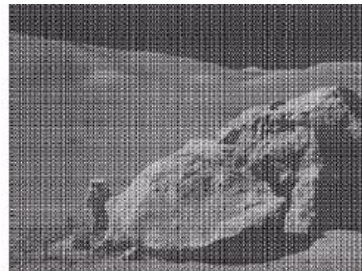
Bandreject filter

- *Gaussian Bandreject filter* is given by:

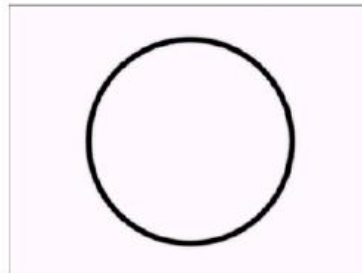
$$H(u, v) = 1 - e^{-\frac{1}{2} \left[\frac{D^2(u, v) - D_0^2}{D(u, v)W} \right]^2}$$

- W is the width of the band
- D_0 is the radial center
- $D(u, v)$ distance from the origin.

Image corrupted by
sinusoidal noise



Butterworth Bandreject
Filter (n=4)



Spectrum of
corrupted image



Filtered Image



Image Example



Before filtering



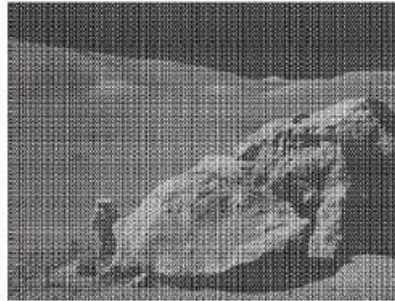
After filtering

Bandpass filters

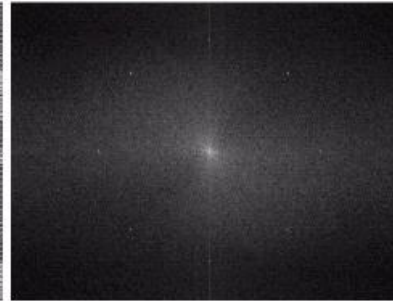
• Bandpass filters perform the opposite function of the bandreject filters and the filter transfer function of a **bandpass filter** is given by:

$$H(u, v)_{bp} = 1 - H(u, v)_{br}$$

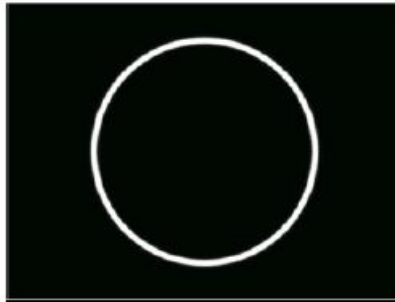
Image corrupted by
sinusoidal noise



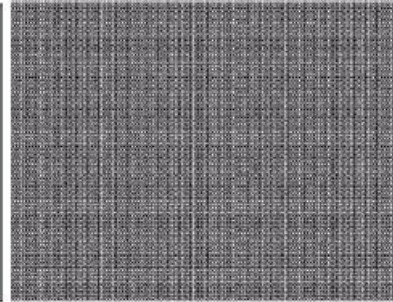
Spectrum of
corrupted image



Butterworth Bandpass
filter



Noise Image



Notch filters

- **Notch filters** rejects/passes frequencies in a predefined neighborhoods about the center frequency.
- Notch filters appear in symmetric pairs due to the symmetry of the Fourier transform.
- The transfer function of **ideal notch filter** of radius of D_0 , with centers at (u_0, v_0) and by symmetry at $(-u_0, -v_0)$, is given by:

$$H(u, v) = \begin{cases} 0 & \text{if } D_1(u, v) \leq D_0 \text{ or } D_2(u, v) \leq D_0 \\ 1 & \text{otherwise} \end{cases}$$

$$D_1(u, v) = \left[(u - M/2 - u_0)^2 + (v - N/2 - v_0)^2 \right]^{1/2}$$

$$D_2(u, v) = \left[(u - M/2 + u_0)^2 + (v - N/2 + v_0)^2 \right]^{1/2}$$

Notch filters

•The transfer function of **Butterworth notch filter** of order n and of radius of D_0 , with centers at (u_0, v_0) and by symmetry at $(-u_0, -v_0)$, is given by:

$$H(u, v) = \frac{1}{1 + \left[\frac{D_0^2}{D_1^2(u, v) D_2^2(u, v)} \right]^n}$$

The transfer function of **Gaussian notch filter** of radius of D_0 , with centers at (u_0, v_0) and by symmetry at $(-u_0, -v_0)$, is given by:

$$H(u, v) = 1 - e^{-\frac{1}{2} \left[\frac{D_1^2(u, v) - D_2^2(u, v)}{D_0^2} \right]}$$

